# Rotary Position Embedding (RoPE) - Part 4 Video Notes

Video: RoPE explained and a kernel written from scratch. This follows RMSNorm. The implementation to keep open is `initializeRopeTables`, `ropeKernel`, and `launchRope` in `src/kernels.cu`, plus `prefill` in `src/main.cpp`.

The video milestone implements RoPE before connecting Q/K projections. The current source also applies the same kernel during decode, using each sequence's absolute position in the cache.

## What problem are we solving?

An embedding lookup gives the same learned starting row to a token ID wherever it appears. The model needs to know something about **where** tokens occur. My rough example was “I bought an Apple watch” versus “Watch an Apple I bought”: the token pieces may overlap, but changing their order changes how they relate. That example is just an intuition, not a promise that the tokenizer gives exactly the same pieces or that the sentences have one clear meaning.

Without positional information, attention based only on content has no built-in way to distinguish a simultaneous rearrangement of tokens. In a causal model, the mask and order of processing already matter, but the attention mechanism still benefits from a direct signal about positions. RoPE supplies that signal to attention by rotating **query and key activations** according to their token positions.

The order in the attention sublayer is:

```
hidden states
  -> RMSNorm
  -> learned Q, K, V projections
  -> rotate Q and K with RoPE; leave V alone
  -> Q·K attention scores, causal mask, softmax
  -> weighted sum of V
```

That last distinction is important. We do not rotate the learned `w_q` and `w_k` **weights** from the SafeTensors file. We rotate the prompt-dependent **Q and K vectors produced by those weights**. At this video's milestone the RoPE table builder and kernel were ready, but `prefill` did not yet create Q/K projection buffers or call `launchRope`. The later Q/K/V notes show that connection.

## The two-number picture

Imagine one query has only two coordinates, `(x0, x1)`. At position `p`, RoPE gives this pair an angle `p * ω`, where `ω` is that pair's frequency. The rotation is ordinary 2D geometry:

```
x0' = x0 * cos(p * ω) - x1 * sin(p * ω)
x1' = x0 * sin(p * ω) + x1 * cos(p * ω)
```

We apply the same kind of rotation to a key at its own position. Position `0` has angle `0`, so its pair is unchanged. Positions in this code are **zero-based**: for a `T`-token prompt, the host currently creates `[0, 1, ..., T-1]` with `std::iota`.

My “I walk my dog” picture still helps. If “I” is at position 1 and “dog” is at position 4 in a simplified numbering, the relevant key and query get angles `1ω` and `4ω`. Their comparison depends on the difference `3ω`. If we shift *both* positions forward by two, the angle difference stays `3ω`. We do not explicitly subtract positions in the kernel; the dot product of the rotated vectors produces the relationship.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

## Why does the dot product see relative distance?

Let `R(a)` be the 2D rotation matrix for angle `a`. A query `q` at position `m` becomes `R(mω)q`; a key `k` at position `n` becomes `R(nω)k`. Their dot product is:

```
(R(mω)q) · (R(nω)k)
  = qᵀ R(mω)ᵀ R(nω) k
  = qᵀ R((n - m)ω) k
```

The key fact is `R(a)ᵀ = R(-a)`. The comparison therefore depends on the **relative offset** `n - m` for that pair. RoPE uses an absolute position to rotate each vector, but the resulting Q/K score contains relative-position information. It does **not** make word order irrelevant or guarantee that two rearranged sentences mean the same thing.

For causal self-attention, a query at `m` may attend to keys at `n <= m`. The causal mask handles that permission separately; RoPE does not replace it.

### The two-number example from my working notes

Let the query be `q = (2, 1)` at position `2`, and the key be `k = (3, 4)` at position `5`. Pick an easy fake frequency `ω = 0.1` radians per token:

```
query angle = 2 * 0.1 = 0.2
key angle   = 5 * 0.1 = 0.5

q' = (2*cos(0.2) - 1*sin(0.2), 2*sin(0.2) + 1*cos(0.2))
   ≈ (1.7615, 1.3774)

k' = (3*cos(0.5) - 4*sin(0.5), 3*sin(0.5) + 4*cos(0.5))
   ≈ (0.7150, 4.9486)

q' · k' ≈ 8.0758
angle difference = (5 - 2) * 0.1 = 0.3
```

That `8.0758` is **one pair's contribution** to a raw Q/K dot product, not a final attention probability. In a full head, the contributions of all pairs are added, the score is divided by `sqrt(head_size)`, the causal mask is applied, and softmax turns allowed scores into attention weights. Also, this particular `query=2, key=5` comparison would be masked in a causal model because the key is in the future. I kept the numbers because they make the rotation clear; swap the positions for an allowed query/key comparison.

## A real Llama head has 64 values, not two

Our query head has `64` numbers, so it contains `32` independent 2D rotation pairs. Llama's layout in this kernel is **half split**:

```
head = [q0, q1, ..., q31, q32, q33, ..., q63]
pairs: (q0, q32), (q1, q33), ..., (q31, q63)
```

This is easy to confuse with adjacent pairs `(q0, q1)`, `(q2, q3)`, etc. The kernel explicitly uses `second_index = first_index + head_size / 2`, so use the half-split picture when reading this code. Each pair in the same token uses that token's position, but each pair has a **different frequency**.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

Here's my tiny four-number head example, with fake frequencies chosen just to see what happens:

```
Q at position 3 = [2, 5, 3, 7]
pair 0 = (2, 3), frequency 0.1  -> angle 3 * 0.1  = 0.3
pair 1 = (5, 7), frequency 0.01 -> angle 3 * 0.01 = 0.03

rotate pair 0: (2, 3) -> (1.024, 3.457)
rotate pair 1: (5, 7) -> (4.788, 7.147)

rotated Q ≈ [1.024, 4.788, 3.457, 7.147]
```

Notice how the results go back into their **original half-split columns**. We are not concatenating the pairs as `(1.024, 3.457, 4.788, 7.147)`. In the real model this happens for all 32 pairs of each query and key head.

## Where the real frequencies come from

For pair index `i` from `0` to `31`, the unscaled inverse frequency is:

```
ω_i = 1 / (theta ^ (2i / head_size))
theta = 500000
head_size = 64
angle(position, i) = position * ω_i
```

That is what the `inverse_frequencies` loop in `initializeRopeTables` calculates. Pair 0 has `ω_0 = 1`, so it turns relatively fast as position rises. Later pairs have smaller frequencies and turn more slowly. The fake `0.1` and `0.01` above are for learning the math; they are not the two values this Llama configuration uses for its first two pairs.

One full turn is `2π` radians, so a frequency `ω` has wavelength:

```
λ = 2π / ω       (measured in token positions)
```

Large frequency means short wavelength and faster rotation. Small frequency means long wavelength and slower rotation. This is the clean version of the formulas that got mangled in my rough notes.

## Llama 3.2's frequency scaling in this code

The table builder applies Llama's long-context scaling to each `ω_i` before calculating sine and cosine. The code uses:

```
original context length = 8192
scale factor = 32
low-frequency factor = 1
high-frequency factor = 4

low-frequency wavelength boundary  = 8192 / 1 = 8192
high-frequency wavelength boundary = 8192 / 4 = 2048
```

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

For each pair's **original** wavelength `λ = 2π / ω`:

| Wavelength | Scaled frequency ω' | What happens |
|---|---|---|
| `λ < 2048` | `ω` | Fast pairs keep their local detail. |
| `2048 <= λ <= 8192` | `(1 - s) * (ω / 32) + s * ω`, with `s = (8192 / λ - 1) / 3` | Blend smoothly between the two behaviors. |
| `λ > 8192` | `ω / 32` | Slow pairs turn even more slowly. |

At `λ = 2048`, `s = 1`, so the blend equals the original frequency. At `λ = 8192`, `s = 0`, so it equals `ω / 32`. The inequalities in the C++ branches put the exact boundaries in the middle branch, but the values join smoothly. After scaling, the table angle is `position * ω'`.

This scaling is part of the model configuration, not a property of every RoPE implementation. Also, the current program builds tables for only `MAX_SEQUENCE_LENGTH = 2048` positions. A long-context frequency formula does not by itself make this checkout support arbitrarily long prompts.

## Why precompute cosine and sine tables?

The same position and pair frequency are reused across heads and layers. Instead of having every GPU thread compute `cos` and `sin` from scratch, `initializeRopeTables` calculates tables on the **CPU**:

```
cos_table[position, pair] = cos(position * scaled_frequency[pair])
sin_table[position, pair] = sin(position * scaled_frequency[pair])
```

Each table has shape `[max_sequence_length, 32]` in FP32. The function allocates two GPU buffers, copies the CPU tables to them, and returns their GPU pointers through `float **cos_table_gpu` and `float **sin_table_gpu`. The pointers are freed in `main` at the end.

The caller must provide position IDs between `0` and `max_sequence_length - 1`. `launchRope` validates pointer and shape arguments, but it does not check every position ID against the table size. Keeping the prompt within the allocated table is therefore part of using this kernel correctly.

## Reading the RoPE kernel from the thread's point of view

The input is a flattened BF16 projection buffer. For each token, its heads are contiguous, and each head has `head_size` values. One thread owns **one pair**, writes its two rotated BF16 results in place, and does not need to synchronize with other threads.

First, the kernel gives a thread a unique pair number:

```
const int work_index = blockIdx.x * blockDim.x + threadIdx.x;
const int pairs_per_head = head_size / 2;
const int pairs_per_token = projection_size / 2;
const int total_pairs = token_count * pairs_per_token;

if (work_index >= total_pairs) return;
```

`projection_size` is **not** the head size. It is the width of a token's whole Q or K projection. For this model, Q has 32 heads × 64 = `2048` values; K has 8 heads × 64 = `512` values. So Q has 1,024 pairs per token, while K has 256. The launcher uses 256 threads per block and ceiling division to cover the full buffer. The bounds check protects the last partially filled block.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

Next, flattening is undone to find the token, head, and pair:

```
const int token_index = work_index / pairs_per_token;
const int pair_in_token = work_index % pairs_per_token;
const int head_index = pair_in_token / pairs_per_head;
const int pair_index = pair_in_token % pairs_per_head;

const int head_base =
    token_index * projection_size + head_index * head_size;
const int first_index = head_base + pair_index;
const int second_index = first_index + pairs_per_head;
```

For example, with a Q projection, `work_index = 33` is token 0, head 1, pair 1. It rotates columns 65 and 97 of that token's 2,048-value Q buffer: `head_base = 64`, then `64+1` and `64+1+32`. Both belong to the same head.

The position ID chooses the table row, and the pair chooses its column:

```
const int position = position_ids[token_index];
const int table_index = position * pairs_per_head + pair_index;
const float cosine = cos_table[table_index];
const float sine = sin_table[table_index];
```

Finally, the kernel converts the two BF16 inputs to FP32, rotates them, and writes BF16 outputs back **in place**:

```
input[first_index] = __float2bfloat16(x0 * cosine - x1 * sine);
input[second_index] = __float2bfloat16(x0 * sine + x1 * cosine);
```

The FP32 intermediate helps precision in the multiply/add steps. The output is rounded to BF16 because that is this engine's activation format. No other thread writes this pair, so there is no block-wide barrier like RMSNorm needed.

## How the host connects RoPE to Q and K

`main` calls `initializeRopeTables` with a 2,048-position limit. `prefill` builds zero-based `position_ids_cpu`, copies them to `position_ids_gpu`, gathers embeddings, and runs the first RMSNorm. At the Part 4 milestone, the RoPE tables and position IDs were ready but `prefill` did not yet call `launchRope`. The current code now applies it to the separate Q and K projection outputs with their respective projection widths. Applying RoPE to `normalized_gpu` would be wrong.

With those projections connected, the mental model is:

```
for each token and each Q/K head:
    for each of the 32 half-split pairs:
        get the token's position and the pair's scaled frequency
        rotate that pair
then attention compares rotated Q against rotated K
```

The dot products are where relative positions show up. The causal mask, grouped-query head mapping, softmax, and mixing of V happen later in attention. RoPE gets the position information into Q/K; it is not the entire attention operation.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.